

Unit 1: Assignment 3

C1. Without running the code, predict the output and justify your answer:

```
int a = 10, b = 20, c = 30;
if (a > b)
    if (b > c)
        printf("Case 1");
    else
        printf("Case 2");
else
    printf("Case 3");
```

(Tests understanding of dangling else)

C2. A student writes:

```
float x = 1 / 3;
printf("%f", x);
```

The output is 0.000000 instead of 0.333333. Identify **two** separate bugs/reasons causing this and correct the code.

C3. Write a single `if` condition (no nested `if`) to check whether a given year is a leap year (a year is a leap year if divisible by 4, but not by 100, unless also divisible by 400).

C4. Explain what happens if you use `=` instead of `==` inside an `if` condition. Give an example where this bug goes **unnoticed** by the compiler (no warning/error) but produces a wrong logical result.

C5. Given the code below, identify the logical flaw (not syntax error) and rewrite it correctly:

```
int marks = 85;
if (marks > 90)
    printf("Grade A");
if (marks > 80)
    printf("Grade B");
if (marks > 70)
    printf("Grade C");
```

(Why does this print more than one grade, and how would you fix it using else-if?)

C6. A user enters 3.9 for an integer variable declared as `int x; scanf("%d", &x);`. What value does `x` actually store, and why? What happens to the leftover input, and how could this leftover affect a subsequent `scanf("%f", &y)` call?

C7. Predict the output and explain the reasoning behind operator precedence and short-circuit evaluation:

```
int a = 0, b = 5, c;
c = (a != 0) && (b / a > 1);
printf("%d", c);
```

Why doesn't this cause a divide-by-zero runtime error?

C8. Design a truth-table based question: Write a program that takes three boolean-like integers (0 or 1) as input representing switches, and turns a bulb ON only if **exactly two** switches are ON (not zero, one, or three). Use only `if-else` and logical operators (no loops).

C9. A junior programmer claims `char` can never store a negative number. Prove or disprove this with a code example and explanation of how `char` is stored in memory (signed vs unsigned).

C10. Without executing, determine the final value of `x` and explain each step:

```
int x = 5;
x = x++ + ++x;
printf("%d", x);
```

(Discuss why this is undefined behavior in C and why different compilers may give different answers.)

C11. The Fall-Through Trap. Predict the exact output, then explain why:

```
int x = 2;
switch (x) {
    case 1: printf("One ");
    case 2: printf("Two ");
    case 3: printf("Three "); break;
    default: printf("Other ");
}
```

Most beginners expect only "Two" to print. Explain in your own words what "fall-through" means, why it happens here, and rewrite the code with the `break` statements needed to make it print only "Two".

C12. Why Can't `switch` Use Floats or Strings? Explain why the following code fails to compile:

```
float x = 2.5;
switch (x) {
    case 2.5: printf("Two point five"); break;
    default: printf("Other");
}
```

Your explanation should mention *how* `switch` is typically implemented internally (a jump table based on integral values), and why this makes floating-point or string case labels fundamentally incompatible with it — not just "against the rules" but *why* the rule exists. What data types **are** legal for `switch` expressions and case labels in C?

C13. Dangling Else Strikes Again — Nested `if` Edition. Predict the output for `marks = 60`, then explain the bug:

```
if (marks >= 50)
    if (marks >= 75)
        printf("Distinction");
    else
        printf("Fail");
```

Most students expect `marks = 60` (which is ≥ 50 but < 75) to print nothing, or some third message — but it actually prints "Fail". Explain exactly which `if` the `else` binds to, and rewrite the code using braces so that a mark of 60 correctly prints "Pass" while under 50 prints "Fail" and 75+ prints "Distinction".

C14. Combined Validation — Menu-Driven Calculator. Design a `switch`-based calculator (+ - * /) that also handles **two edge cases** most students forget:

1. Division by zero (must not just crash or print `inf`/garbage — detect and print an error message)
2. An invalid menu choice / operator character (must be handled by `default`, not silently ignored)

Write the full logic (`switch` structure with these two safeguards) and explain, for each safeguard, what would go wrong in the program if that specific check were removed.